

LOWERBOUND — Project Documentation

Oct 2, 2026 · @Abhi

Overview

LOWERBOUND schedules a college's weekly classes with no section or teacher clashes, using the fewest classrooms the week allows, and proves no schedule can use fewer. On the university sample (27 sections across six branches) it cuts 27 rooms to 20, which equals the proven minimum, so the result is optimal.

Problem statement (DAA hackathon): University Timetable Room Minimizer. Place every class in a weekly period so that no section attends two classes at once and no teacher teaches two classes at once, and minimize the number of classrooms.

The pitch: colleges give every section its own room, so rooms sit empty whenever a section is out. We reschedule the same classes into the fewest rooms the week allows, and prove it.

The proof in one line: every class needs a room in its period, so rooms $\geq$ [total classes $\div$ periods per week]. When our schedule hits that number, no schedule can do better.

Sample result	Value
Sections	27 in six branches: CSE 7 (G and H combined), CSE-DS 6, CSM 8, ECE 3, IoT/R&AI 1, BBA 2
Classes per week	702 (27 sections $\times$ 26), 24 subjects, 49 teachers
Periods per week	36 (6 days $\times$ 6 periods)
Floor (lower bound)	$\lceil 702 \div 36 \rceil = 20$ rooms
Rooms today (one per section)	27
Rooms after minimizing	20, gap 0, so optimal
Empty room-periods per week	270 $\rightarrow$ 18
Classrooms freed	7
Clashes	0, checked by <code>Engine.verify</code>

What the app does:

- States the result up front (702 classes, 36 periods, 20 rooms, lower bound 20, gap 0, verified) and animates the 27-room baseline into the 20-room timetable.

- Shows the reduction to graph coloring and proves the lower bound with live numbers and an interactive pigeonhole slider.
- Draws the real conflict graph and steps through every DSatur choice the solver makes.
- Compares four algorithms on the same input, and shows the engine's own repair (ejection chains) and polish events.
- Explains room assignment, verifies the result live (including deliberately broken copies), proves feasibility and optimality, and states the complexity.
- Handles disruptions (a teacher out, a room closed, a section away) by moving only the affected classes, without re-solving, and writes ready-to-send notices.
- Accepts other data: a random college generator or pasted CSV.

Where the code is: [Supercalifragilisticexpialidociouscoder/DAA_Hackathon](https://supercalifragilisticexpialidociouscoder/DAA_Hackathon), branch `lowerbound-app` (commit `62b0e91`). The pull request into `main` has not been opened yet.

Quick start

The app needs only Node and npm; everything runs in the browser, with fonts bundled so it works offline on stage. It was built and tested with Node 22.16 and npm 10.9.

Command	What it does
<code>npm install</code>	Installs React, Vite, TypeScript, Vitest and the two font packages
<code>npm run dev</code>	Starts the dev server at <code>http://localhost:5173</code> (Vite's default port)
<code>npm test</code>	Runs the 18 Vitest tests: engine parity and disruption invariants
<code>npm run build</code>	Type-checks (<code>tsc -b</code>) and builds the static site into <code>dist/</code>
<code>npm run preview</code>	Serves the built <code>dist/</code> locally
<code>npm run typecheck</code>	Type-checks only

The build output is a static site: any static host can serve `dist/`. The solver runs in the visitor's browser, so there is no server or API.

Two-minute stage demo

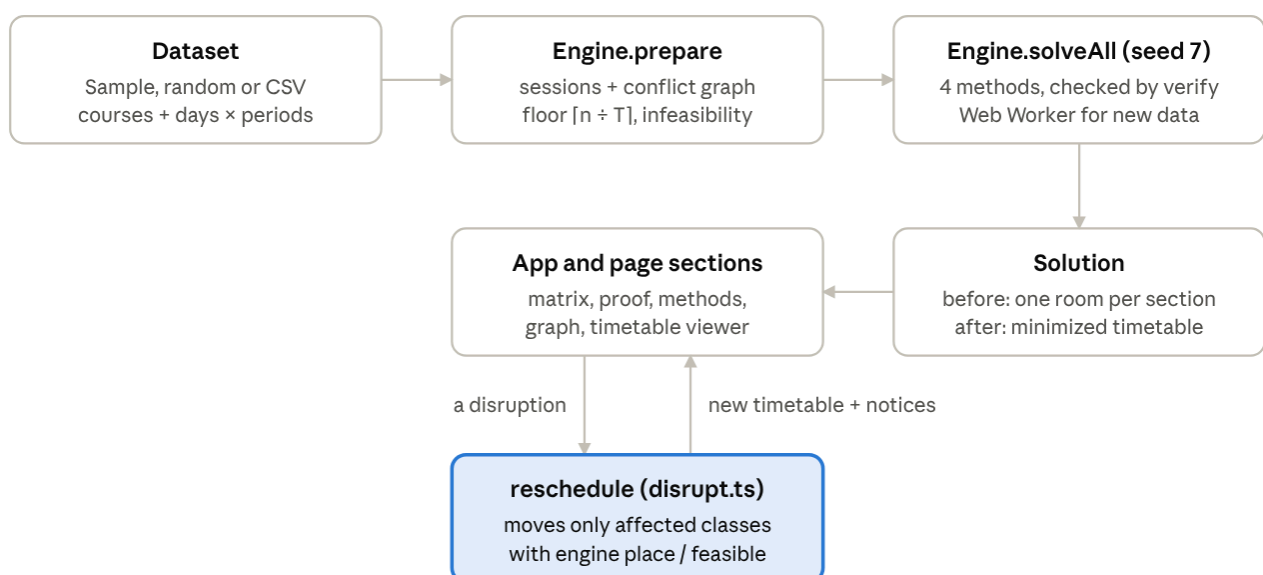
1. **Open the page.** "Today" shows one room per section: 27 rooms, 270 empty room-periods as pink hatching, and a yellow dashed line at the floor of 20 rooms.
2. **Click Minimize rooms.** Every class slides to its new period and room, rows R21 to R27 collapse, the counter rolls $27 \rightarrow 20$ and a chalk circle draws around the 20.
3. **Scroll to Why 20 is the floor.** Read the bound off the live numbers: 19 rooms could hold only 684 classes, 18 short of 702. Point at the verification checks.
4. **Click Pick a teacher.** Ask a judge to name one. They're marked absent on their busiest day; only the affected classes move and are ringed in yellow. Rooms stay at 20, clashes stay at 0, and the Changes panel shows copy-ready notices.
5. **Take questions** with the method comparison, the conflict graph and the timetable viewer.

The demo is deterministic: the solver seed is fixed at 7, so the sample gives the same timetable on every machine.

Architecture

A dataset flows one way: the engine turns it into a solved timetable, the UI draws it, and disruptions edit that timetable in place rather than solving again. The stack is Vite 8, React 19 and TypeScript 5.9, tested with Vitest 5, with no UI kit and no server.

One dataset flows through the engine; disruptions edit the result in place



LOWERBOUND data flow · 6 parts

The top row runs once per dataset; the loop at the bottom runs once per disruption and never goes back through the solver.

Data flow, step by step:

1. A `Dataset` is `{ name, courses, D, P }`: a list of courses (section, subject, faculty, weekly hours) plus days and periods per day.
2. `Engine.prepare` builds the `Problem`: one session per class hour, conflict-graph degrees, edge count, the lower bound `lb` and any infeasibility messages.
3. `Engine.solveAll(pb, 7)` runs the four methods and returns their slots, rooms and timings.
4. `buildSolution` keeps what the UI needs: `before` (fill-in-order slots, one room per section), `after` (our minimized timetable) and subject short codes.
5. `App` holds the solution and a history of disruptions. The current timetable is the latest disruption's result, or `after` if there is none.

The sample is solved synchronously at start-up (about 5 ms), so the page renders with data immediately. Datasets from the Change data dialog are solved in a Web Worker so a hard instance never freezes the page; the main thread then re-runs `prepare`, which is cheap and deterministic.

Files

Path	Role
<code>src/engine/engine.ts</code>	TypeScript port of <code>engine.js</code> : <code>prepare</code> , <code>solveAll</code> , <code>verify</code> , <code>mulberry32</code> , plus its state helpers and stage functions exported unchanged
<code>src/engine/data.ts</code>	Port of <code>data.js</code> : <code>sample</code> (the university dataset), <code>original</code> (the 245-class sample), <code>random</code> , <code>parseCSV</code> , <code>toCSV</code>
<code>src/engine/trace.ts</code>	Visualization layer: traced DSatur, repair and polish with their event logs, replay helpers
<code>src/engine/disrupt.ts</code>	Incremental repair: blocked slots and reinsertion with ejection chains
<code>src/engine/solver.worker.ts</code>	Web Worker that runs <code>prepare</code> + <code>solveAll</code> off the main thread
<code>src/engine/types.ts</code>	Shared types: <code>Course</code> , <code>Dataset</code> , <code>Problem</code> , <code>Schedule</code> , method results

Path	Role
<code>src/lib/solution.ts</code>	Builds the UI's <code>Solution</code> , explains failed solves, picks sync or worker solving
<code>src/lib/ring.ts</code>	Ring layout, hit-testing and neighbour lists for the graph views
<code>src/lib/demos.ts</code>	The harder instances used by the repair and polish sections
<code>src/lib/labels.ts</code> , <code>notices.ts</code> , <code>hooks.ts</code>	Labels and subject codes, notice text, small React hooks
<code>src/components/Hero.tsx</code>	Title, result strip, pipeline rail, the Run LOWERBOUND animation
<code>src/components/Model.tsx</code> , <code>Bound.tsx</code>	Sections 1 and 2: the coloring model and the lower bound
<code>src/components/ConflictGraph.tsx</code> , <code>Playground.tsx</code>	Sections 3 and 4: the graph explorer and the DSatur stepper
<code>src/components/Methods.tsx</code> , <code>Repair.tsx</code> , <code>Polish.tsx</code> , <code>Rooms.tsx</code>	Sections 5 to 8
<code>src/components/Verification.tsx</code> , <code>Complexity.tsx</code>	Sections 9 and 10
<code>src/components/Incremental.tsx</code> , <code>Disruptions.tsx</code>	Section 11: disruption controls, matrix and Changes panel
<code>src/components/Timetable.tsx</code> , <code>Technical.tsx</code>	Sections 12 and 13
<code>src/components/Matrix.tsx</code> , <code>Counter.tsx</code> , <code>Section.tsx</code> , <code>DataDialog.tsx</code>	Room × period matrix, counter and chalk circle, section frame and pseudocode, Change data dialog
<code>src/styles/app.css</code>	Both themes, tokens, layout, responsive rules
<code>reference/engine.js</code> , <code>reference/data.js</code>	The original tested files, kept for the parity and trace tests
<code>tests/*.test.ts</code>	Parity, dataset, trace and disruption suites

The algorithm engine

Picking periods is graph coloring: classes are vertices, clashes are edges, periods are colors, and the rooms needed equal the size of the biggest color class. Our method colors with DSatur, then repairs the busiest periods with ejection chains until it reaches the pigeonhole floor.

Problem model

- **Session:** each weekly hour of a course is one class to place. A course with 5 hours becomes 5 sessions.
- **Slot:** a period of the week, numbered $t = \text{day} \times P + \text{period}$, so there are $T = D \times P$ slots (36 on the sample).
- **Clash:** two sessions clash if they share a section (same students) or a faculty member (same teacher).
- **Rooms needed:** the largest number of classes in any one slot, since each running class needs its own room.
- **Degree:** a session's clash count is $\text{secCount} + \text{facCount} - \text{both} - 1$, where **both** counts sessions with the same section and teacher. Edges = sum of degrees $\div 2$ (12,345 on the sample).
- **Infeasibility:** `prepare` reports any section or teacher with more classes than the week has periods.

The lower bound

If every class needs a room in its slot, the busiest slot holds at least the average, rounded up:

$$\text{\text{rooms}} \geq \left\lceil \frac{n}{T} \right\rceil = \left\lceil \frac{702}{36} \right\rceil = 20$$

Nineteen rooms could hold at most $19 \times 36 = 684$ classes, 18 short of 702, so some period must run 20 classes. The bound counts classes only, not clashes; it can be unreachable, but whenever a schedule meets it, that schedule is optimal.

The four methods

Method	Order	Period choice	Complexity	Sample rooms
Fill in order (baseline)	Section by section, cycling through that section's subjects	Earliest free period, avoiding the same subject twice a day	$O(n \cdot T)$	27
Largest-first greedy (Welsh-Powell)	Most clashes first	Least busy free period; a same-day repeat counts as +1.5 load	$O(n \log n + n \cdot T)$	20
DSatur	Next = the class with the fewest periods still open, ties by most clashes	Least busy free period	$O(n^2 + E)$	20
DSatur + repair (ours)	DSatur first pass	Then repair, polish and room assignment	$O(n^2 + E) + \text{local search}$	20

Our pipeline

1. **DSatur first pass.** On this sample it already gives 20 rooms, the floor.
2. **Repair (ejection chains).** While the busiest period is above the floor, take a class out of a fullest period and move it to a period at least 2 below the peak. If exactly one class blocks it there, push that blocker to a third period (a 2-move chain). On this sample: 0 moves, because the first pass is already at the floor. On the original 245-class sample (kept in reference/) it took 4 moves, $8 \rightarrow 7$ rooms.
3. **Polish.** Move classes so no subject meets twice in one day, never letting any period exceed the room count; a blocked move may swap with one blocker. On this sample: 0 moves, 0 repeats left.
4. **Seeded restarts.** If the floor or zero repeats isn't reached, DSatur reruns with randomized tie-breaks (`mulberry32` , seed 7) until a 450 ms budget, hard-capped at 1.8 s. The sample hits the floor on the first pass.
5. **Room assignment.** Within each period, a section keeps the room it used the period before that day, else its home room, else the lowest free room. On the sample: 85 room changes a week, about 3.1 per section. CSE sections mostly stay in one room; ECE,

IoT/R&AI and BBA move between many, because the home-room rule (section number mod room count) puts them on rooms CSE sections already hold. The baseline uses one fixed room per section.

Verification

`Engine.verify(pb, slot, room)` walks every class and returns the first problem it finds (`class v unplaced`, `section clash at t`, `faculty clash at t`, `room clash at t`) or `null` when there is none. The UI runs it on whatever timetable is on screen, and the tests run it on every result.

Disruption mode

When something changes mid-week, LOWERBOUND moves only the classes that have to move, never re-solves, and keeps the room count. On the sample, every "teacher out for a whole day" case ($49 \text{ teachers} \times 6 \text{ days} = 294 \text{ cases}$) keeps 20 rooms with zero clashes, moving 2.6 classes on average and 6 at most, in a few milliseconds each.

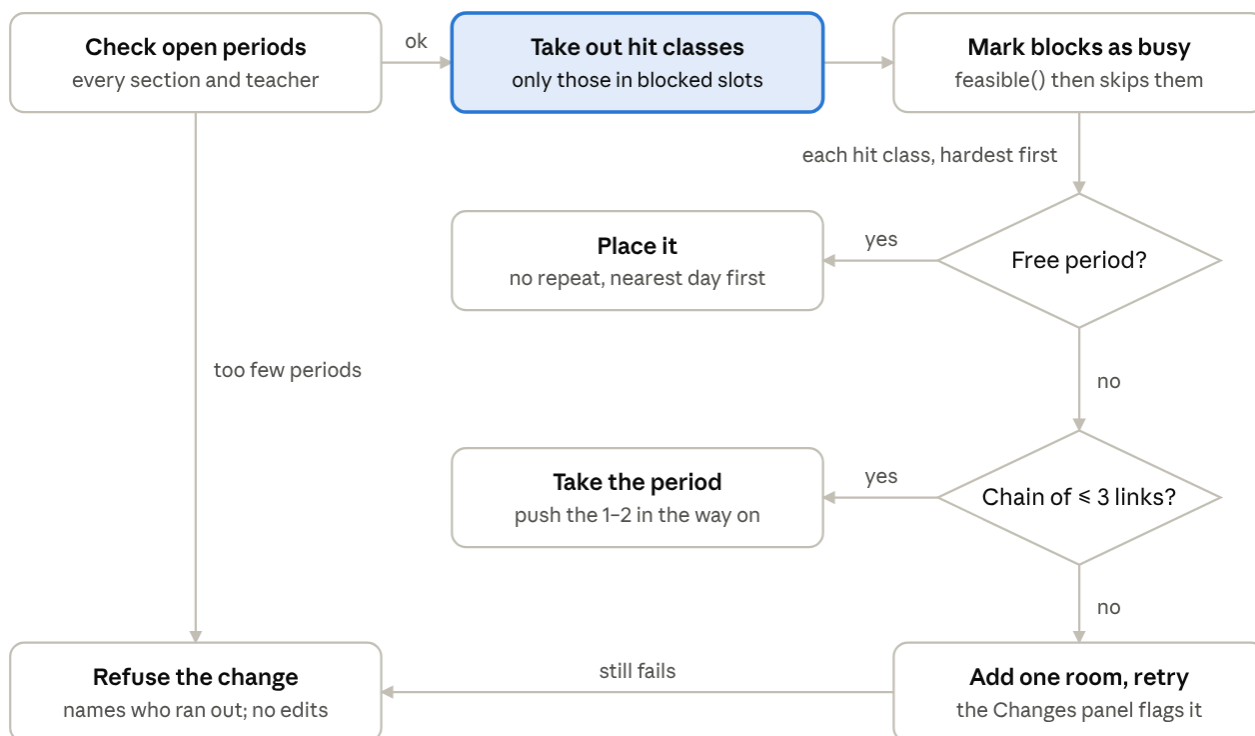
Kinds of disruption

Kind	Blocks	Example
Teacher unavailable	That teacher in the chosen periods	Dr. Harish Kumar out Monday
Room closed	That room in the chosen periods	R3 closed Tuesday, periods 2-4
Section away (trip, event)	That section in the chosen periods	CSE-B away Friday

Each disruption is `{ kind, who, day, periods }`, where an empty `periods` list means the whole day. They are stored as blocked slots in three bitmaps laid out like the engine's own busy maps: `section × slot`, `teacher × slot` and `room × slot`.

How a disruption is handled (`reschedule` in `disrupt.ts`)

Only hit classes move; an extra room is the last resort



`reschedule()` in `disrupt.ts` · 6 steps, 2 decisions

Most disruptions end at the first or second decision; on the sample, no teacher-out case ever reached the extra-room step.

- 1. Check it can work.** Every section and teacher needs at least as many open periods as classes. If not, it stops with a reason, such as "CSE-A has 26 classes a week, but only 24 periods stay open", and nothing changes.
- 2. Load the current timetable** into the engine's state with its own `place` function.
- 3. Take out only the hit classes:** those sitting in a period now blocked for their section or teacher. For a closed room, only periods that now have more classes than open rooms lose a class, and it's the one in the closed room.
- 4. Mark blocked periods busy** in the engine's section and teacher busy maps, so the engine's own `feasible` check keeps everyone out of them.
- 5. Re-place each class, most constrained first** (fewest open periods, then most clashes). Each class first tries a free period. Failing that, it tries ejection chains of growing depth (1, 2, then 3 links): take a period anyway and push the one or two classes in the way elsewhere, the same move the repair step uses. Every edit is journaled, so a failed chain is rolled back exactly. A node budget of 40,000 per depth bounds the search.

6. **Fallback:** if a class fits nowhere, try once with one extra room. If it still doesn't fit, the disruption is refused with the section and teacher named. The UI says plainly when an extra room was needed.
7. **Rooms:** a class that kept its period keeps its room unless that room is closed. Moved classes prefer their section's room in the neighbouring period, then the room the section uses most.

Priorities, in order: zero clashes, no more rooms than before, fewest classes moved. Among free periods, a class prefers one without a same-subject repeat that day, then a day close to its original, then a period with a room to spare.

Stacking, undo and output

- Disruptions stack: each new one runs on the current timetable with every block so far.
- **Undo last** steps back one disruption; **Back to the minimized timetable** clears them all. Both animate the affected classes home.
- Each result reports classes moved (split into hit directly and shifted to make space), rooms, `Engine.verify` result, room-only changes and solve time.
- Notices are generated per section and per teacher, ready to paste into a group chat:

```
CSE-D: timetable change
Dr. Harish Kumar is unavailable on Monday.
• DE (Dr. Harish Kumar): Mon P3 in R4 → Fri P4 in R19
All other classes stay as they are.
```

Measured results on the sample

Scenario	Classes moved	Rooms	Clashes
Any teacher out any whole day (294 cases)	2.6 on average, 6 at most (6 cases move nothing: no class that day)	20 in every case	0
Dr. Harish Kumar out Monday (first name in Pick a teacher)	5 (4 hit, 1 shifted to make space)	20	0
Any section away any whole day (162 cases)	5.5 on average, 10 at most	20 in every case	0
Any room closed any whole day (120 cases)	3.5 on average, 8 at most	20 in every case	0
CSE-A away Monday and Tuesday	refused: 26 classes, 24 open periods	unchanged	unchanged

The website, section by section

The page reads like a project defence: the result first, then the model, the proof of the bound, the graph, the algorithm step by step, the improvements, verification, complexity, the extension, and the final timetable. Every number and event on it is computed from the live data.

Hero: the result

- **Title:** the LOWERBOUND wordmark and "University timetable optimization using graph coloring and DSatur", with one line on the model.
- **Result strip:** classes $n = 702$ (vertices), periods $T = 36$ (colors), rooms used 20 (largest color class), lower bound 20 ($\lceil n \div T \rceil$), gap 0 (optimal). Below it: "Optimal solution, verified" with the live `Engine.verify(timetable) → null`.
- **Pipeline rail:** Input → Conflict graph → Graph coloring → DSatur → Repair → Room assignment → Verification → Minimum rooms, each linking to its section.
- **Run LOWERBOUND:** the room × period matrix starts at the baseline (fill in order, one room per section: 27 rooms) and animates into the solved timetable (20 rooms). Columns are colors, so a column's height is its color-class size. The counter rolls 27 → 20 and a chalk circle draws round the 20. Afterwards a Baseline / LOWERBOUND toggle and Replay appear.

The numbered sections

#	Section	What you can do
1	The problem is graph coloring	Read the class → vertex, clash → edge, period → color, rooms → largest color class mapping and the formal model; toggle three real classes between their edges and a valid coloring
2	Why 20 is the minimum	See $702 \div 36 = 19.50\dots$ rounded up to 20 and $19 \times 36 = 684$ being 18 short; drag a room-count slider over a pigeonhole grid that marks each count impossible, open or achieved
3	The conflict graph	Hover or tap any of the 702 dots to draw its real section and teacher edges; read its degree split (for example $25 + 18 - 3 = 40$); show all 12,345 edges; walk vertices with the arrow keys
4	DSatur, step by step	Play, step or scrub all 702 choices: the vertex picked, its saturation and degree, ties, its available periods, the period chosen, and the busiest period so far, on the graph and a day × period grid, beside pseudocode
5	Same input, different algorithm	Click Fill in order, Welsh–Powell, DSatur or LOWERBOUND to compare rooms, gap, time, complexity, pseudocode and each one's classes-per-period profile against the bound
6	Repair: ejection chains	Step through the engine's repair events as a chain (class → blocked period → ejected class → third period → result), with the busiest period after each event
7	Polish	Step through the engine's polish moves, seeing the section's day before and after with the repeated subject marked
8	Room assignment	See color-class size per period; click a period to list its classes and their rooms
9	Verification and proof	Read the formal checks, run <code>Engine.verify</code> live, break the timetable on purpose (double-book a section, teacher or room) to watch <code>verify</code> report it, and read the feasible-vs-optimal proof

#	Section	What you can do
10	Complexity	Variables with their values here, DSatur's $O(n^2 + E)$ derivation, space $O(n + T \cdot (S + F) + C \cdot D)$, and every stage with its measured time
11	Incremental repair	Mark a teacher, room or section unavailable; only affected classes move on the matrix (ringed, with dashed outlines where they were). See affected, moved, untouched, rooms, verification and copy-ready notices
12	The final timetable	View by section, teacher or room; copy the week as CSV
13	Technical details	Architecture, testing, limitations and the repository link

The header keeps links to the main sections, the dataset name, **Change data** and the theme toggle.

How the visualizations stay honest

- **DSatur steps** come from a step-recording copy of the engine's DSatur in `trace.ts`. It calls the engine's own feasibility and period-choice functions, and a test checks it colors every vertex exactly like `Engine.solveAll` on five datasets.
- **Repair and polish events** come from the engine's own `repair()` and `polish()`, run with their built-in event logs switched on.
- **On the university sample** DSatur already reaches 20, so those two stages make no moves, and the sections say so. A picker switches to the original 245-class sample (4 repair moves; 2 polish swaps removing 2 repeats) or a random 16-section week, where repair performs a real ejection chain and goes from 14 to the floor of 13.
- **The verification buttons** break a copy of the timetable and run the real `verify` on it.

Data inputs

The Change data dialog offers three sources: the built-in sample, a seeded random college, or pasted CSV. Each shows a live preview (sections, teachers, classes, periods and the floor) and any infeasibility before you apply; applying solves in a Web Worker and resets the hero to "today".

University sample

One semester across six branches: 27 sections over 6 days × 6 periods, 49 fictional faculty. CSE-G and CSE-H are always taught together, so they are one section, CSE-G+H. The data model has no separate notion of combined sections (a section is one group of students who attend together), so a combined group is simply one section.

Branch	Sections	Subjects (hours a week)
CSE	A, B, C, D, E, F, G+H	P&S 5, DE 5, DAA 5, BWD 5, ACS 3, LRAS 3
CSE-DS	A-F	the same six
CSM	A-H	the same six
ECE	A, B, C	Electronic Circuit Analysis (ECA) 5, Analog and Digital Communications (ADC) 5, Linear and Digital IC Applications (LDICA) 4, Electromagnetic Fields and Waves (EMFW) 4, Signals and Systems (S&S) 4, Probability Theory and Stochastic Processes (PTSP) 4
IoT/R&AI	1 section	Sensors and Actuators (S&A) 5, Embedded Systems (ES) 5, Introduction to Robotics (ITR) 4, Artificial Intelligence Fundamentals (AIF) 4, Wireless Sensor Networks (WSN) 4, Control Systems for Robotics (CSR) 4
BBA	A, B	Financial Management (FM) 5, Marketing Management (MM) 5, Human Resource Management (HRM) 4, Organisational Behaviour (OB) 4, Business Statistics (BS) 4, Business Communication (BC) 4

Every section has 26 classes a week, 702 in all. Each P&S, DE, DAA and BWD teacher takes 3 of the 21 CSE/CSE-DS/CSM sections, grouped differently per subject; ACS and LRAS trainers take 3–5. Seven teachers work across branch groups (two ECE faculty in IoT/R&AI, a DE and a P&S teacher in ECE, a maths teacher in ECE and BBA, a CSM DAA teacher in IoT/R&AI, an English trainer in BBA). Loads run from 4 to 19 classes a week. The original 245-class sample is kept in `reference/data.js` for the engine parity test.

Random college

Slider	Range	Meaning
Sections	4–24	Named Sec-A, Sec-B, ...
Classes per section a week	12–36	Split across 4–9 subjects (about hours ÷ 4)
Sections per teacher	1–4	Each teacher takes this many consecutive sections of a subject
Days	5–6	Mon–Fri or Mon–Sat
Periods a day	4–8	
Shuffle seed	1–99	Same seed and settings give the same college; Next steps the seed

Subjects come from a pool of 16 and teacher names are generated, both from the seed.

Paste CSV

One row per subject per section: section, subject, faculty, weekly hours. A header row is optional; lines starting with `#` and blank lines are skipped. Days (5 or 6) and periods a day (4–8) are chosen beside the text box, and **Fill with the sample** loads the sample as CSV to edit.

```
section,subject,faculty,hours
CSE-A,Design & Analysis of Algorithms,Dr. K. Srinivas Rao,5
CSE-A,Operating Systems,Mr. B. Ravi Kumar,5
CSE-B,Design & Analysis of Algorithms,Dr. K. Srinivas Rao,5
CSE-B,Operating Systems,Ms. G. Sravani,5
```

Values are split on commas, so a subject or name can't contain a comma.

Errors and failures

Situation	Message shown	Apply
Too few values on a row	"Line 4 has 3 values; it needs section, subject, faculty, hours."	Disabled
Bad hours	"Line 5: hours must be a whole number from 1 to 40."	Disabled
Empty field	"Line 6 has an empty section, subject or faculty."	Disabled

Situation	Message shown	Apply
More classes than periods	"Sec-A needs 40 classes but the week only has 36 periods." (same for a teacher)	Disabled
Solver gets stuck	"No clash-free timetable found. The solver ran out of periods for Sec-B's VLSI class with Ms. A. Sravani..." plus a suggestion	Dialog stays open, current data kept

Line numbers match the line in the text box, blank and comment lines included. The "solver gets stuck" case is real: 24 sections $\times$ 20 classes into 5 days $\times$ 4 periods with 4 sections per teacher passes the count checks, but DSatur cannot finish it.

Design system

The look is an Indian college classroom: a green chalkboard with painted room-number plates, plus a light "green ledger paper" twin. It deliberately avoids glassmorphism, gradient blobs, generic SaaS cards, dashboard templates and ALL-CAPS eyebrow labels. Boldness is spent in one place, the $9 \rightarrow 7$ animation; everything else is quiet.

Color tokens

All colors are CSS custom properties on `:root`, swapped by `data-theme` on `<html>`. Dark (chalkboard) is the default; the toggle's choice is saved in `localStorage` and applied before first paint.

Token	Chalkboard (dark)	Ledger paper (light)	Used for
<code>--bg</code>	<code>#17221E</code>	<code>#EDF1E6</code>	Page background (light adds faint ruled lines)
<code>--panel</code>	<code>#1D2A25</code>	<code>#F7F9F2</code>	Board and panels
<code>--line</code>	<code>#2E3F38</code>	<code>#C9D6C6</code>	Rules and borders
<code>--ink</code>	<code>#E6EBE2</code>	<code>#1B2822</code>	Main text (chalk)
<code>--muted</code>	<code>#93A39A</code>	<code>#55665C</code>	Secondary text
<code>--accent</code>	<code>#F1CF55</code>	<code>#F1CF55</code>	Yellow chalk: main buttons, highlight fills

Token	Chalkboard (dark)	Ledger paper (light)	Used for
<code>--accent-strong</code>	#F1CF55	#846800	Floor line, chalk circle, moved rings (darkened on paper for contrast)
<code>--waste</code>	#EF8C9C	#B0445C	Pink chalk: empty room-periods, excess rows, errors
<code>--plate-bg / --plate-ink</code>	#E6EBE2 / #17221E	#1D2A25 / #EDF1E6	Painted room plates

Section colors: a box of colored chalk

Twelve muted pastels on the board, and deeper versions of the same hues on paper.

Section `i` uses color `i mod 12`, so colors repeat past 12 sections.

#	Board	Paper		#	Board	Paper
0	#8FB8DE blue	#3F78A8		6	#A3AEEA periwinkle	#5363B8
1	#E8B07A orange	#C0712E		7	#D2D98A lime	#7F8B22
2	#A8D08D green	#5E8F3E		8	#D3A1C4 mauve	#94577E
3	#C9A2D9 lilac	#8B5BA6		9	#9DD3E8 sky	#2D8DB3
4	#7FCBBE teal	#2F8C7E		10	#E09F86 terracotta	#AB5A43
5	#E3C9A0 sand	#A07F45		11	#B3DDC0 mint	#3F9467

Type

- **Big Shoulders Stencil Display** (700, 800) for room plates, big numbers, the equation and the logo, like stencil-painted room numbers.
- **Schibsted Grotesk** (400–700) for everything else.
- Both are bundled through `@fontsource`, so nothing loads from the network.
- Tabular numerals are applied only where numbers line up (stats, tables, plates). Applied to whole paragraphs, Schibsted Grotesk's tabular setting also widens commas and full stops.

Motion

Animation	Timing
Minimize sweep	680 ms per block, start staggered 0–460 ms left to right
Counter roll and stat count-down	900 ms, starting at 150 ms
Extra rows collapse	At 1,180 ms, 340 ms height and opacity
Chalk circle draws	At 1,480 ms, 760 ms stroke draw
Before/After toggle	520 ms, staggered 0–240 ms
Disruption moves	720 ms each, 70 ms apart; other blocks dim for 2.8 s

Movement uses only CSS transforms, which the browser can run on the GPU. With `prefers-reduced-motion`, the JavaScript skips every sequence and the CSS sets all transitions to 0 ms.

Accessibility and responsive behavior

- Visible focus rings (2 px, accent) on everything focusable, and a skip link to the demo.
- The matrix is keyboard-navigable with a live region; the counter and stats carry text for screen readers; the conflict graph has an `aria-label` summary and focusable section arcs.
- Text colors meet contrast for their size on both themes (for example, muted text is about 6:1 on the board).
- Breakpoints at 1,100, 1,000, 760 and 520 px. Below 1,000 px the tally moves above the matrix; below 520 px it becomes a compact counter beside the stats.
- The matrix's cell width adapts from 12 to 34 px; when it can't fit, the matrix scrolls sideways inside its panel. The page itself has no horizontal scroll at 375 px.

Testing and verification

All 39 automated tests pass (`npm test`, about 1 second), the production build is clean, and the full demo flow was checked by hand in a browser at desktop and phone widths.

Automated tests

File	Test	What it proves
<code>parity.test.ts</code>	Original sample (<code>reference/data.js</code>)	TypeScript and original JS give identical sessions, bound, edges and schedules; rooms are 9, 8, 8, 7 and the floor is 7
<code>parity.test.ts</code>	University sample	Both engines give identical schedules on the 702-class dataset
<code>parity.test.ts</code>	6 random colleges	Identical generated data and identical schedules, from 4 to 24 sections
<code>parity.test.ts</code>	CSV	Same parsed courses as the original; line numbers point at the pasted line; <code>toCSV</code> round-trips the sample
<code>sample.test.ts</code>	Structure	The 27 sections in six branches, in order; CSE-G+H is one section, with no separate G or H
<code>sample.test.ts</code>	Subjects	CSE, CSE-DS and CSM take exactly the six; ECE, IoT/R&AI and BBA take their own, used by no other branch; one row per section and subject
<code>sample.test.ts</code>	Short codes	P&S, DE, DAA, BWD, ACS, LRAS, and every code unique
<code>sample.test.ts</code>	Shared teachers	Every CSE/CSE-DS/CSM teacher has at least 2 sections; at least 3 teach across branches; edges exceed section-only edges
<code>sample.test.ts</code>	Sessions	One distinct session per class hour
<code>sample.test.ts</code>	Solve	Floor = $\lceil n \div T \rceil$; ours equals it and is below one-room-per-section; <code>verify</code> null; 0 repeats; 27 sections, 702 classes, floor 20, 20 rooms
<code>sample.test.ts</code>	Determinism	Seed 7 gives the same periods and rooms twice

File	Test	What it proves
<code>sample.test.ts</code>	Pick a teacher	Every teacher out on their busiest day moves at least one class, keeps the room count and 0 clashes
<code>disrupt.test.ts</code>	Every teacher out every day (294)	All succeed, 0 clashes, never an extra room
<code>disrupt.test.ts</code>	Every section away every day (162)	All succeed, no extra room
<code>disrupt.test.ts</code>	Every room closed every day (120)	All succeed, 0 clashes
<code>disrupt.test.ts</code>	Specific periods only	Only the chosen periods are affected
<code>disrupt.test.ts</code>	Four stacked disruptions	Each step stays valid on top of the last
<code>disrupt.test.ts</code>	Determinism	Same disruption, same result
<code>disrupt.test.ts</code>	Impossible block	Refused with "CSE-A has 26 classes a week, but only 24 periods stay open"
<code>disrupt.test.ts</code>	3 random colleges	Valid results or a clear reason

`trace.test.ts` adds 12 more:

- `Data.original()` matches `reference/data.js` exactly.
- On five datasets, the traced DSatur colors every vertex exactly like the engine.
- The traced pipeline equals `solveAll`'s first pass, and its repair and polish logs replay to the same loads and the same final timetable.
- The original sample really exercises repair (8 → 7) and polish.

Every successful disruption is checked for: `Engine.verify` returns `null`; no class sits in a blocked period or closed room; every class not reported as moved is exactly where it was; room numbers stay in range.

The parity test compares the original `reference/engine.js` against the TypeScript port. On hard random instances the solver's retry loop depends on time, so "ours" is compared only when both runs did the same number of retries; the other three methods are always compared.

Checked by hand in the browser

- University sample: 27 → 20 with gap 0, the circle draws, Before/After and Replay work, `Engine.verify` shows `null`.
- Pick a teacher (Dr. Harish Kumar, Monday): 5 moved, 20 rooms, 0 clashes, about 4 ms; notices correct.
- Room closed (R3, Tuesday periods 2–4, on the original sample): 3 moved, closed cells drawn cross-hatched.
- Random 12 sections: 12 → 10 rooms, the floor. Random 24 sections × 30 classes: 24 → 18 rooms, the floor.
- Bad CSV: correct inline errors, Apply disabled. Stuck solve: dialog stays open with the explanation.
- 375 px wide: no page-level horizontal scroll; the matrix scrolls inside its panel.
- Light theme, hover tooltips and keyboard navigation of the matrix.

Not verified: the frame rate (the browser pane was hidden during testing, which throttles animation) and the reduced-motion path (the setting couldn't be simulated). Both should be checked once on the demo machine.

Limitations, decisions and judge questions

The main limits are honest ones: the floor only proves optimality when we reach it, and disruption handling minimizes moves heuristically, not provably.

Known limitations

- **The floor can be unreachable.** It counts classes, not clashes. When the gap is above 0, the app says so and does not claim optimality.
- **Hard random instances depend on machine speed.** The solver retries until a time budget runs out, so the number of retries, and possibly the result, can differ between machines. The sample is unaffected: it reaches the floor on the first pass.
- **Fewest moves is a heuristic.** Each displaced class uses the shortest ejection chain available at that moment; the total across all displaced classes is not proven minimal.
- **CSV parsing is simple.** Values are split on commas, so fields can't contain commas or quotes.
- **Section colors repeat** after 12 sections; tooltips and the timetable viewer tell them apart.
- **Not yet verified on the demo machine:** frame rate and the reduced-motion path.
- **Room size isn't modelled.** Every room fits any class, so the combined CSE-G+H class takes one room like any other section.

- **Some sections change rooms a lot.** The room step keeps a section in the room it used the period before, else $\text{room number} = \text{section number} \bmod \text{room count}$. CSE sections mostly stay put, but ECE, IoT/R&AI and BBA land on rooms CSE sections already hold and move between 11 to 13 rooms a week (85 room changes in total).

Design decisions

Decision	Why
Keep the engine's logic identical and prove it with a parity test	The original was already tested; the port must not change results
One deliberate change: CSV errors use the line number as pasted	The original counted after dropping blank lines, so its numbers pointed at the wrong line
Disruptions reuse the engine's <code>place</code> , <code>unplace</code> and <code>feasible</code> , with blocks stored as busy bits	Same feasibility rules as the solver, with no second implementation to drift
Never re-solve on a disruption	A re-solve could move most of the week; people need a short list of changes
Sample solved on load; other data solved in a Web Worker	The sample appears instantly; large data never freezes the page
Fixed seed (7)	The live demo behaves the same every time
Fonts bundled, no network calls	Works offline on stage

Likely judge questions

- **How do you know 20 is optimal?** By pigeonhole: $36 \text{ periods} \times 19 \text{ rooms}$ holds only 684 of the 702 classes, so some period must run 20 at once. We use exactly 20.
- **Why DSatur?** It's the classic coloring heuristic: placing the most constrained class first avoids painting itself into a corner. On this sample its first pass already reaches 20. On the original 245-class sample it reached 8 and the repair step closed the last gap.
- **What's NP-hard here?** Choosing periods is coloring the conflict graph with T colors while keeping the largest color class small, a bounded coloring problem that is NP-hard in general. That's why we pair a heuristic with a provable bound.
- **What is an ejection chain?** Moving a class into a period that's blocked by one other class, and pushing that class somewhere else; chains can be several links long.

- **What happens when a teacher is absent?** Only their classes in that period move, plus any class pushed aside to make room. Rooms stay the same and both the section and the teacher get a notice.
- **Is it deterministic?** Yes for the sample and for any data that reaches the floor on the first pass; the seed is fixed at 7.
- **Does it need a server?** No. Everything runs in the browser, including the solver.